Python Review

Python + NumPy + Matplotlib

Contents

Environment Setup 2

Python Basics 2

- Variables 2
- Python Collections (Arrays) 7
  - Lists 7
  - Tuple 11
  - Sets 12
  - Dictionaries 14
- If-Else 17
- While loop 18
- For loop 19
- Functions 20

NumPy 22

Matplotlib 27

Summary 34

Exercise 34

NOTE THIS LAB IS COMPLETELY BASED ON PYTHON TUTRIOALS PROPOSED IN https://www.w3schools.com/

Environment Setup

- Anaconda environment: https://www.anaconda.com/
- IDE: Pycharm with Jupyter, Spyder, JupyterLab, Jupyter Notebook.
    - Recommended: Spyder version 4.2.4 with notebook plugin.
    - To install notebook plugin for Spyder use:
        - From Conda environment, open CMD
        - Run the command:

$$conda\ install\ spyder - notebook\ \ - c\ spyder - ide$$

Python Basics

- Python is a popular programming language.
- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.).
- Python has a simple syntax similar to the English language.
- Python can be treated in a procedural way, an object-oriented way or a functional way.
- It is used for:
    - Web Development (server-side),
    - Software Development,
    - Mathematics,
    - System Scripting.
    - Data Science

Variables

- Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.

```
x = 5
y = "John"
print(x)
print(y)
```

- If you want to specify the data type of a variable, this can be done with casting.

```
x = str(3)     # x will be '3'
y = int(3)     # y will be 3
z = float(3)   # z will be 3.0
```

- You can get the data type of a variable with the $type()$ function.

```
x = 5
y = "John"
print(type(x))
print(type(y))
```

- Python allows you to assign values to multiple variables in one line:

```
x, y, z = "Orange", "Banana", "Cherry"
print(x)
print(y)
print(z)
```

- If you have a collection of values in a list, tuple etc. Python allows you extract the values into variables. This is called unpacking.

```
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits
print(x)
print(y)
```

- You can also use the + character to add a variable to another variable:

```
x = "Python is "
y = "awesome"
z =  x + y
print(z)
```

- For numbers, the + character works as a mathematical operator:

```
x = 5
y = 10
print(x + y)
```

- You can assign a multiline string to a variable by using three quotes:

```
a = """Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua."""
print(a)
```

- Square brackets can be used to access elements of the string.

```
a = "Hello, World!"
print(a[1])
```

- Since strings are arrays, we can loop through the characters in a string, with a for loop.

```
for x in "banana":
    print(x)
```

- To get the length of a string, use the $len()$ function.

```
a = "Hello, World!"
print(len(a))
```

- To check if a certain phrase or character is present in a string, we can use the keyword in.

```
txt = "The best things in life are
free!"
print("free" in txt)
```

- To check if a certain phrase or character is NOT present in a string, we can use the keyword $not\ in$.

```
txt = "The best things in life are free!"
print("expensive" not in txt)
```

- Python Arithmetic Operators

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*****	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
******	Exponentiation	x ** y
//	Floor division	x // y

- Python Comparison Operators

Operator	**Name**	**Example**
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

- Python Logical Operators

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

- Python Identity Operators

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

- Python Bitwise Operators

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

Python Collections (Arrays)

	Tuple	List	Dictionary	Set
Eample	('Book 1', 12.99)	['apple', 'banana', 'orange']	{'name': 'Joe', 'age': 10}	{10, 20, 12}
Mutable?	Immutable	Mutable	Mutable	Mutable
Ordered?	Ordered	Ordered	Preserves order since Python 3.7	Unordered
Iterable?	Yes (takes linear time)	Yes (takes linear time)	Yes (constant time)	Yes (constant time)
Use case	Immutable data	Data that needs to change	Key/Value pairs	Unique items

Lists

- Lists are used to store multiple items in a single variable.

```
thislist = ["apple", "banana","cherry"]
print(thislist)
```

- To determine how many items a list has, use the $len()$ function:

```
thislist = ["apple", "banana","cherry"]
print(len(thislist))
```

- List items can be of any data type:

```
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]
list3 = [True, False, False]
list4 = ["abc", 34, True, 40, "male"]
```

- List items are indexed and you can access them by referring to the index number:

```
thislist = ["apple", "banana","cherry"]
print(thislist[1])
```

- Negative indexing means start from the end

```
thislist = ["apple", "banana","cherry"]
print(thislist[-1])
```

- You can specify a range of indexes by specifying where to start and where to end the range. The return value will be a new list with the specified items.

```
thislist = ["apple", "banana",
"cherry", "orange", "kiwi", "melon",
"mango"]
print(thislist[2:5])
```

  - Note: The search will start at index 2 (included) and end at index 5 (not included).
- By leaving out the start value, the range will start at the first item:

```
thislist = ["apple", "banana",
"cherry", "orange", "kiwi", "melon",
"mango"]
print(thislist[:4])
```

- By leaving out the end value, the range will go on to the end of the list:

```
thislist = ["apple", "banana",
"cherry", "orange", "kiwi", "melon",
"mango"]
print(thislist[2:])
```

- To determine if a specified item is present in a list use the in keyword:

```
thislist = ["apple", "banana",
"cherry"]
if "apple" in thislist:
    print("Yes, 'apple' is in the
fruits list")
```

- To change the value of a specific item, refer to the index number:

```
thislist = ["apple", "banana", "cherry"]
thislist[1] = "blackcurrant"
print(thislist)
```

- Change a Range of Item Values

```
thislist = ["apple", "banana", "cherry",
"orange", "kiwi", "mango"]
thislist[1:3] = ["blackcurrant",
"watermelon"]
print(thislist)
```

- The $insert()$ method inserts an item at the specified index:

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(2, "watermelon")
print(thislist)
```

- To add an item to the end of the list, use the $append()$ method:

```
thislist = ["apple", "banana", "cherry"]
thislist.append("orange")
print(thislist)
```

- To append elements from another list to the current list, use the $extend()$ method.

```
thislist = ["apple", "banana", "cherry"]
tropical = ["mango", "pineapple",
"papaya"]
thislist.extend(tropical)
print(thislist)
```

- The $remove()$ method removes the specified item.

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist)
```

- The $pop()$ method removes the specified index.

```
thislist = ["apple", "banana", "cherry"]
thislist.pop(1)
print(thislist)
```

- If you do not specify the index, the $pop()$ method removes the last item.

```
thislist = ["apple", "banana", "cherry"]
thislist.pop()
print(thislist)
```

- The $clear()$ method empties the list. The list still remains, but it has no content.

```
thislist = ["apple", "banana", "cherry"]
thislist.clear()
print(thislist)
```

Tuple

- A tuple is a collection which is ordered and unchangeable.

```
thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

- To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

```
thistuple = ("apple",)
print(type(thistuple))

#NOT a tuple
thistuple = ("apple")
print(type(thistuple))
```

- Access methods for tuples are the same as lists

✓ CHECKPOINT: how to modify the items of a tuple (delete, insert) although it is immutable?

- Unpacking a tuple:

```
fruits = ("apple", "banana", "cherry")

(green, yellow, red) = fruits

print(green)
print(yellow)
print(red)
```

- If the number of variables is less than the number of values, you can add an * to the variable name and the values will be assigned to the variable as a list:

```
fruits = ("apple", "banana", "cherry",
"strawberry", "raspberry")

(green, yellow, *red) = fruits

print(green)
print(yellow)
print(red)
```

- If the asterisk is added to another variable name than the last, Python will assign values to the variable until the number of values left matches the number of variables left.

```
fruits = ("apple", "mango", "papaya",
"pineapple", "cherry")

(green, *tropic, red) = fruits

print(green)
print(tropic)
print(red)
```

Sets

- A set is a collection which is unordered, unchangeable, and unindexed.

```
thisset = {"apple", "banana", "cherry"}
print(thisset)
```

- Duplicate values will be ignored:

```
thisset = {"apple", "banana", "cherry",
"apple"}
print(thisset)
```

- You cannot access items in a set by referring to an index or a key.
    - But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the in keyword.

```
thisset = {"apple", "banana", "cherry"}

for x in thisset:
    print(x)
```

```
thisset = {"apple", "banana", "cherry"}

print("banana" in thisset)
```

- To add one item to a set use the $add()$ method.

```
thisset = {"apple", "banana", "cherry"}

thisset.add("orange")

print(thisset)
```

- To add items from another set into the current set, use the $update()$ method.

```
thisset = {"apple", "banana", "cherry"}
tropical = {"pineapple", "mango",
"papaya"}

thisset.update(tropical)

print(thisset)
```

- To remove an item in a set, use the $remove()$, or the $discard()$ method.

```
thisset = {"apple", "banana", "cherry"}

thisset.remove("banana")

print(thisset)
```

```
thisset = {"apple", "banana", "cherry"}

thisset.discard("banana")

print(thisset)
```

Dictionaries

- Dictionaries are used to store data values in key: $value$ pairs.
    - A dictionary is a collection which is ordered, changeable and does not allow duplicates.

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
print(thisdict)
```

- Dictionaries cannot have two items with the same key:
    - Duplicate values will overwrite existing values:

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964,
    "year": 2020
}
print(thisdict)
```

- You can access the items of a dictionary by referring to its key name, inside square brackets:

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
x = thisdict["model"]
```

- There is also a method called $get()$ that will give you the same result:

```
x = thisdict.get("model")
```

- The $keys()$ method will return a list of all the keys in the dictionary.

```
x = thisdict.keys()
```

- You can change the value of a specific item by referring to its key name:

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
thisdict["year"] = 2018
```

- The $update()$ method will update the dictionary with the items from the given argument.

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
thisdict.update({"year": 2020})
```

- Adding an item to the dictionary is done by using a new index key and assigning a value to it:

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
thisdict["color"] = "red"
print(thisdict)
```

- The $pop()$ method removes the item with the specified key name:

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
thisdict.pop("model")
print(thisdict)
```

If-Else

```
a = 200
b = 33
if b > a:
        print("b is greater than a")
elif a == b:
        print("a and b are equal")
else:
        print("a is greater than b")
```

- One line if statement

```
if a > b: print("a is greater than b")
```

- Shorthand $If \ldots Else$

```
a = 2
b = 330
print("A") if a > b else print("B")
```

- The and keyword is a logical operator, and is used to combine conditional statements:

```
a = 200
b = 33
c = 500
if a > b and c > a:
        print("Both conditions are True")
```

- The *or* keyword is a logical operator, and is used to combine conditional statements:

```
a = 200
b = 33
c = 500
if a > b or a > c:
        print("At least one of the
conditions is True")
```

While loop

- With the *while* loop we can execute a set of statements if a condition is true.

```
i = 1
while i < 6:
        print(i)
        i += 1
```

- With the *break* statement we can stop the loop even if the while condition is true:

```
i = 1
while i < 6:
        print(i)
        if i == 3:
                break
        i += 1
```

- With the continue statement we can stop the current iteration, and *continue* with the next:

```
i = 0
while i < 6:
        i += 1
        if i == 3:
                continue
        print(i)
```

For loop

- A *for* loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
        print(x)
```

- The *range*() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

```
for x in range(6):
        print(x)
```

- Using the start parameter:

```
for x in range(2, 6):
        print(x)
```

- Increment the sequence with 3 (default is 1):

```
for x in range(2, 30, 3):
        print(x)
```

- The *else* keyword in a for loop specifies a block of code to be executed when the loop is finished:

```
for x in range(6):
        print(x)
else:
        print("Finally finished!")
```

Functions

- In Python a function is defined using the *def* keyword
- To call a function, use the function name followed by parenthesis:

```
def my_function():
        print("Hello from a function")

my_function()
```

- Information can be passed into functions as arguments.

```
def my_function(fname):
        print(fname + " Refsnes")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

- If you do not know how many arguments that will be passed into your function, add a $*$ before the parameter name in the function definition.

```
def my_function(*kids):
        print("The youngest child is " +
kids[2])

my_function("Emil", "Tobias", "Linus")
```

- You can also send arguments with the $key = value$ syntax. This way the order of the arguments does not matter.

```
def my_function(child3, child2, child1):
        print("The youngest child is " +
child3)

my_function(child1 = "Emil", child2 =
"Tobias", child3 = "Linus")
```

- If you do not know how many keyword arguments that will be passed into your function, add two asterisk: $**$ before the parameter name in the function definition.
    - This way the function will receive a dictionary of arguments, and can access the items accordingly:

```
def my_function(**kid):
        print("His last name is " +
kid["lname"])

my_function(fname = "Tobias", lname =
"Refsnes")
```

- The following example shows how to use a default parameter value. If we call the function without argument, it uses the default value:

```
def my_function(country = "Norway"):
        print("I am from " + country)

my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

NumPy

- NumPy is a Python library used for working with arrays.
- NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
- The array object in NumPy is called $ndarray$, it provides a lot of supporting functions that make working with $ndarray$ very easy.

- NumPy is used to work with arrays. The array object in NumPy is called $ndarray$.
    - We can create a NumPy $ndarray$ object by using the $array()$ function.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)

print(type(arr))
```

- 0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

```
import numpy as np

arr = np.array(42)

print(arr)
```

- 1-D Arrays

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

- 2-D Arrays

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr)
```

- 3-D arrays

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]],
[[1, 2, 3], [4, 5, 6]]])

print(arr)
```

- NumPy Arrays provides the $ndim$ attribute that returns an integer that tells us how many dimensions the array has.

```
a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1,
2, 3], [4, 5, 6]]])

print(a.ndim)
print(b.ndim)
print(c.ndim)
print(d.ndim)
```

- Access 1-D array:

```
arr = np.array([1, 2, 3, 4])

print(arr[0])
```

- Access 2-D Arrays:

```
arr = np.array([[1,2,3,4,5],
[6,7,8,9,10]])

print('2nd element on 1st row: ', arr[0,
1])
```

- Access 3-D Arrays

```
arr = np.array([[[1, 2, 3], [4, 5, 6]],
[[7, 8, 9], [10, 11, 12]]])

print(arr[0, 1, 2])
```

- Use negative indexing to access an array from the end.

```
arr = np.array([[1,2,3,4,5],
[6,7,8,9,10]])

print('Last element from 2nd dim: ',
arr[1, -1])
```

- Slicing in python means taking elements from one given index to another given index.
    - We pass slice instead of index like this: $[start: end]$.
    - We can also define the step, like this: $[start: end: step]$.

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[1:5])
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[1:5:2])
```

- Slicing 2-D Arrays

```
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8,
9, 10]])

print(arr[1, 1:4])
print(arr[0:2, 2])
print(arr[0:2, 1:4])
```

- NumPy arrays have an attribute called $shape$ that returns a tuple with each index having the number of corresponding elements.

```
arr = np.array([[1, 2, 3, 4], [5, 6, 7,
8]])

print(arr.shape)
```

- Generate a random integer from 0 to 100:

```
x = np.random.randint(100)

print(x)
```

- Generate a random float from 0 to 1:

```
x = np.random.rand()

print(x)
```

- Generate a 2-D array with 3 rows, each row containing 5 random integers from 0 to 100:

```
x = np.random.randint(100, size=(3, 5))

print(x)
```

- Generate a 2-D array with 3 rows, each row containing 5 random numbers:

```
x = np.random.rand(3, 5)

print(x)
```

✓ TASK: what is $random.seed()$ function in NumPy?

Matplotlib

- Matplotlib is a low-level graph plotting library in python that serves as a visualization utility.
- The $plot()$ function is used to draw points (markers) in a diagram. By default, the $plot()$ function draws a line from point to point.
- The function takes parameters for specifying points in the diagram.
    - Parameter 1 is an array containing the points on the x-axis.
    - Parameter 2 is an array containing the points on the y-axis.

- If we need to plot a line from $(1, 3)$ to $(8, 10)$, we have to pass two arrays $[1, 8]$ and $[3, 10]$ to the plot function.

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints)
plt.show()
```

- To plot only the markers, you can use shortcut string notation parameter 'o', which means 'rings'.

```
xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints, 'o')
plt.show()
```

- You can plot as many points as you like, just make sure you have the same number of points in both axis.

```
xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints, 'o')
plt.show()
```

- If we do not specify the points in the x-axis, they will get the default values 0, 1, 2, 3, etc.

```
ypoints = np.array([3, 8, 1, 10, 5, 7])

plt.plot(ypoints)
plt.show()
```

- You can use the keyword argument $marker$ to emphasize each point with a specified marker:

```
ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, marker = 'o')
plt.show()
```

  - Other markers: https://matplotlib.org/stable/api/markers_api.html
- You can use the keyword argument $linestyle$, or shorter ls, to change the style of the plotted line:

```
ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, linestyle = ':')
plt.show()
```

  - Other line styles: https://matplotlib.org/stable/gallery/lines_bars_and_markers/linestyles.html

- You can use the keyword argument $color$ or the shorter c to set the color of the line:

```
ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, color = 'r')
plt.show()
```

  - List of color sets:
    https://matplotlib.org/stable/gallery/color/named_colors.html
    https://matplotlib.org/stable/tutorials/colors/colors.html

- You can use the keyword argument $linewidth$ or the shorter lw to change the width of the line.

```
ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, linewidth = '20.5')
plt.show()
```

- You can plot as many lines as you like by simply adding more $plt.plot()$ functions:

```
y1 = np.array([3, 8, 1, 10])
y2 = np.array([6, 2, 7, 11])

plt.plot(y1)
plt.plot(y2)

plt.show()
```

- You can also plot many lines by adding the points for the x- and y-axis for each line in the same $plt.plot()$ function.

```
x1 = np.array([0, 1, 2, 3])
y1 = np.array([3, 8, 1, 10])
x2 = np.array([0, 1, 2, 3])
y2 = np.array([6, 2, 7, 11])

plt.plot(x1, y1, x2, y2)
plt.show()
```

- With $Pyplot$, you can use the $xlabel()$ and $ylabel()$ functions to set a label for the x- and y-axis.
- you can use the $grid()$ function to add grid lines to the plot.
- You can use the $title()$ function to set a title for the plot.

```
x = np.array([80, 85, 90, 95, 100, 105,
110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280,
290, 300, 310, 320, 330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)

plt.grid()

plt.show()
```

- Display multiple plots with the subplots() function

```
#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1, 2, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)
plt.plot(x,y)

plt.show()
```

  - The $subplots()$ function takes three arguments that describes the layout of the figure.
  - The layout is organized in rows and columns, which are represented by the first and second argument.
  - The third argument represents the index of the current plot.

- You can draw as many plots you like on one figure, just describe the number of rows, columns, and the index of the plot.

```
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 3, 1)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 3, 2)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 3, 3)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 3, 4)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 3, 5)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 3, 6)
plt.plot(x,y)

plt.show()
```

- You can add a title to each plot with the $title()$ function, and add a title to the entire figure with the $suptitle()$ function:

```
#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1, 2, 1)
plt.plot(x,y)
plt.title("SALES")

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)
plt.plot(x,y)
plt.title("INCOME")

plt.suptitle("MY SHOP")
plt.show()
```

- you can use the $scatter()$ function to draw a scatter plot.

```
x =
np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y =
np.array([99,86,87,88,111,86,103,87,94,78,
77,85,86])

plt.scatter(x, y)
plt.show()
```

- You can set your own color for each scatter plot with the $color$ or the c argument:

```
x =
np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y =
np.array([99,86,87,88,111,86,103,87,94,78,
77,85,86])
plt.scatter(x, y, color = 'hotpink')

x =
np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,
12])
y =
np.array([100,105,84,105,90,99,90,95,94,10
0,79,112,91,80,85])
plt.scatter(x, y, color = '#88c999')

plt.show()
```

Summary

- We reviewed the basics of Python including variables, collections (list, tuple, dictionary, set), flow control (if…else, while loop, for loop), and functions.
- We introduced NumPy library for creating and manipulating arrays.
- We introduced Matplotlib library for plotting data in different styles.
- The key resource for mastering NumPy and matplotlib libraries is review the official documentation and examples as required based on your problem.

Exercise

1. What are $mean$, $median$, $mode$, $variance$, and $standard\ deviation$?
2. Given the list $data = [15,18,13,20,20,11,14,99,100,200,201,0]$, use NumPy to calculate the mean, median, variance, and standard deviation.
3. What is Gaussian distribution?
4. Using NumPy and matplotlib, plot a figure of a Gaussian distribution of $mean = 6.0$, $standard\ deviation = 1.0$, with $size = 100000$